

DSA Revision Notes

Topic 6 — Trees, BST & Tree DP

Binary Trees · BST · AVL · Traversals · LCA · Diameter
Serialisation · Tree DP · Path Sum · 25+ C++ examples · LeetCode Hard

12

Sections

30+

C++ Examples

10

Tree Diagrams

28+

LeetCode Problems

TOC Table of Contents**1. Introduction — Tree Terminology & Types**

- Node, edge, height, depth, balanced, complete, full, perfect
- Binary Tree vs BST vs AVL vs Red-Black

2. TreeNode Structure & Memory

- Standard struct · Building trees · Level-order construction

3. Tree Traversals — All 5 Types

- Inorder · Preorder · Postorder · Level-order · Zigzag
- Iterative versions using stack/queue
- Morris Traversal — $O(1)$ space

4. Binary Tree — Classic Problems

- Height/Depth · Count nodes · Diameter · Balanced check
- Same tree · Subtree · Invert · Mirror

5. Binary Tree — Views & Boundary

- Left/Right view · Top/Bottom view · Boundary traversal
- Vertical order · Diagonal traversal

6. Lowest Common Ancestor (LCA)

- LCA in BT · LCA in BST · LCA with parent pointers
- Binary Lifting for $O(\log n)$ LCA queries

7. Binary Search Tree (BST) — Full Operations

- Insert · Search · Delete · Floor · Ceil · Rank
- Validate BST · Convert Sorted Array to BST

8. BST — Advanced Problems

- Kth Smallest · Range Sum · Trim BST
- BST to Greater Sum Tree · Two Sum in BST

9. Serialisation & Construction

- Serialize/Deserialize · Build from pre+inorder
- Build from postorder+inorder · Recover BST

10. Path Sum Problems

- Path Sum I/II/III · Max Path Sum · Count paths

11. Tree DP — Patterns & Problems

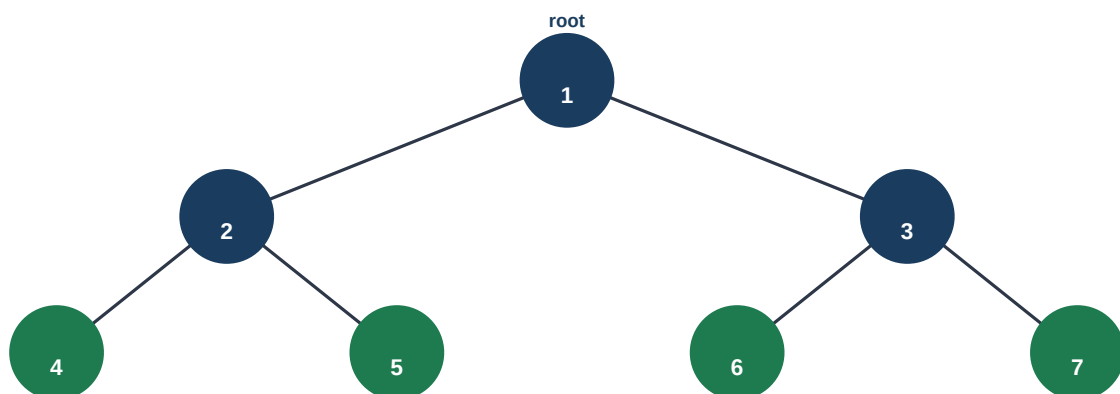
- What is Tree DP · Subtree state aggregation
- House Robber III · Max sum no adjacent · Balanced partition
- Diameter via Tree DP · Longest Zig-Zag path

12. Complexity Cheat Sheet & LeetCode Map

1 Introduction — Tree Terminology & Types

1.1 Core Terminology

- **Node:** Basic unit storing data and pointers to children.
- **Root:** The topmost node with no parent. Entry point to tree.
- **Leaf:** Node with no children (left == right == nullptr).
- **Height of node:** Longest path from node down to a leaf (leaf height = 0).
- **Depth of node:** Distance from root to that node (root depth = 0).
- **Height of tree:** Height of root node. Empty tree has height = -1.
- **Degree:** Number of children a node has. Binary tree: max degree = 2.
- **Level:** Set of nodes at the same depth. Root is at level 0 (or 1).
- **Subtree:** A node and all its descendants form a subtree.
- **Ancestor:** Any node on the path from root to a given node.
- **Sibling:** Nodes that share the same parent.
- **Complete BT:** All levels full except possibly last; last filled left to right.
- **Full BT:** Every node has 0 or 2 children (never 1).
- **Perfect BT:** All internal nodes have 2 children; all leaves at same level.
- **Balanced BT:** Height difference of left/right subtrees at every node is at most 1.



Perfect Binary Tree: all leaves at same depth, all internal nodes have 2 children

1.2 Types of Trees — Comparison

Type	Key Property	Search	Insert	Delete	Height
Binary Tree	At most 2 children per node	$O(n)$	$O(n)$	$O(n)$	$O(n)$ worst

BST	left < node < right (all subtrees)	O(h)	O(h)	O(h)	O(n) unbalanced
AVL Tree	BST + h_left - h_right <= 1	O(log n)	O(log n)	O(log n)	O(log n)
Red-Black Tree	BST + coloring rules → ~balanced	O(log n)	O(log n)	O(log n)	O(log n)
Segment Tree	Range queries in O(log n)	O(log n)	O(log n)	O(log n)	O(log n)
Trie	Prefix tree for strings	O(L)	O(L)	O(L)	O(alphabet)

Table 1: Tree types and their operation complexities

2 TreeNode Structure & Building Trees

2.1 Standard TreeNode (LeetCode style)

C++

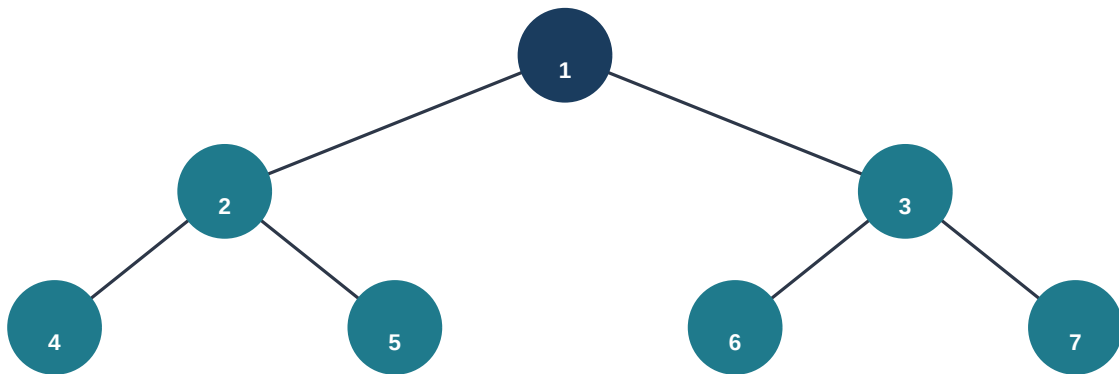
```
1 // Standard definition used in all LeetCode tree problems
2 struct TreeNode {
3     int val;
4     TreeNode* left;
5     TreeNode* right;
6     TreeNode() : val(0), left(nullptr), right(nullptr) {}
7     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
8     TreeNode(int x, TreeNode* l, TreeNode* r) : val(x), left(l), right(r) {}
9 };
10
11 // Build a tree manually
12 //      1
13 //     / \
14 //    2   3
15 //   / \
16 //  4   5
17
18 TreeNode* buildExample() {
19     TreeNode* root = new TreeNode(1);
20     root->left = new TreeNode(2);
21     root->right = new TreeNode(3);
22     root->left->left = new TreeNode(4);
23     root->left->right = new TreeNode(5);
24     return root;
25 }
```

2.2 Build Tree from Level-Order (BFS)

C++

```
1  #include <queue>
2  #include <vector>
3  using namespace std;
4
5  // Build tree from level-order array (LeetCode format)
6  // e.g. {1,2,3,null,null,4,5} → standard level-order tree
7  TreeNode* buildFromLevelOrder(vector<int>& vals) {
8      if (vals.empty() || vals[0] == -1) return nullptr;
9      TreeNode* root = new TreeNode(vals[0]);
10     queue<TreeNode*> q;
11     q.push(root);
12     int i = 1;
13     while (!q.empty() && i < (int)vals.size()) {
14         TreeNode* node = q.front(); q.pop();
15         if (i < (int)vals.size() && vals[i] != -1) {
16             node->left = new TreeNode(vals[i]);
17             q.push(node->left);
18         }
19         i++;
20         if (i < (int)vals.size() && vals[i] != -1) {
21             node->right = new TreeNode(vals[i]);
22             q.push(node->right);
23         }
24         i++;
25     }
26     return root;
27 }
28 // Use -1 to represent null nodes in the input vector
29 // Time: O(n) Space: O(n)
```

3 Tree Traversals — All 5 Types



Example tree for all traversals: root=1

■ Traversal Orders for the Tree Above

Inorder (Left → Root → Right): 4, 2, 5, 1, 6, 3, 7 ← sorted for BST!

Preorder (Root → Left → Right): 1, 2, 4, 5, 3, 6, 7 ← used for tree copy/serialise

Postorder (Left → Right → Root): 4, 5, 2, 6, 7, 3, 1 ← used for deletion / bottom-up DP

Level-order (BFS): 1, 2, 3, 4, 5, 6, 7 ← layer by layer

Reverse level-order: 4, 5, 6, 7, 2, 3, 1 ← bottom-up layers

3.1 Recursive Traversals — $O(n)$ time, $O(h)$ space

C++

```
1 // Inorder: Left → Root → Right
2 void inorder(TreeNode* root, vector<int>& res) {
3     if (!root) return;
4     inorder(root->left, res);
5     res.push_back(root->val);    // visit root BETWEEN children
6     inorder(root->right, res);
7 }
8
9 // Preorder: Root → Left → Right
10 void preorder(TreeNode* root, vector<int>& res) {
11     if (!root) return;
12     res.push_back(root->val);    // visit root FIRST
13     preorder(root->left, res);
14     preorder(root->right, res);
15 }
16
17 // Postorder: Left → Right → Root
18 void postorder(TreeNode* root, vector<int>& res) {
19     if (!root) return;
20     postorder(root->left, res);
21     postorder(root->right, res);
22     res.push_back(root->val);    // visit root LAST
23 }
24 // All: Time O(n) Space O(h) recursive stack [h = O(log n) balanced, O(n) skewed]
```

3.2 Iterative Inorder — using Stack (LC 94)

C++

```
1  vector<int> inorderIterative(TreeNode* root) {
2      vector<int> res;
3      stack<TreeNode*> st;
4      TreeNode* curr = root;
5      while (curr || !st.empty()) {
6          // Go as far left as possible
7          while (curr) {
8              st.push(curr);
9              curr = curr->left;
10         }
11         // Process node
12         curr = st.top(); st.pop();
13         res.push_back(curr->val);
14         // Move to right subtree
15         curr = curr->right;
16     }
17     return res;
18 }
19 // Time: O(n) Space: O(h)
```

3.3 Iterative Preorder — using Stack

C++

```
1  vector<int> preorderIterative(TreeNode* root) {
2      if (!root) return {};
3      vector<int> res;
4      stack<TreeNode*> st;
5      st.push(root);
6      while (!st.empty()) {
7          TreeNode* node = st.top(); st.pop();
8          res.push_back(node->val); // visit
9          // Push RIGHT first so LEFT is processed first
10         if (node->right) st.push(node->right);
11         if (node->left) st.push(node->left);
12     }
13     return res;
14 }
```

3.4 Level-Order (BFS) — using Queue (LC 102)

C++

```
1  vector<vector<int>> levelOrder(TreeNode* root) {
2      if (!root) return {};
3      vector<vector<int>> res;
4      queue<TreeNode*> q;
5      q.push(root);
6      while (!q.empty()) {
7          int levelSize = q.size();          // snapshot: nodes at this level
8          vector<int> level;
9          for (int i = 0; i < levelSize; i++) {
10             TreeNode* node = q.front(); q.pop();
11             level.push_back(node->val);
12             if (node->left) q.push(node->left);
13             if (node->right) q.push(node->right);
14         }
15         res.push_back(level);
16     }
17     return res;
18 }
19 // Time: O(n) Space: O(w) - w = max width of tree
```

3.5 Zigzag Level-Order (LC 103)

C++

```
1  vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
2      if (!root) return {};
3      vector<vector<int>> res;
4      queue<TreeNode*> q;
5      q.push(root);
6      bool leftToRight = true;
7      while (!q.empty()) {
8          int sz = q.size();
9          vector<int> level(sz);
10         for (int i = 0; i < sz; i++) {
11             TreeNode* node = q.front(); q.pop();
12             // Write to correct position based on direction
13             int pos = leftToRight ? i : (sz - 1 - i);
14             level[pos] = node->val;
15             if (node->left) q.push(node->left);
16             if (node->right) q.push(node->right);
17         }
18         res.push_back(level);
19         leftToRight = !leftToRight;    // flip direction each level
20     }
21     return res;
22 }
```

3.6 Morris Traversal — O(1) Space Inorder

Morris traversal threads the tree: use right pointer of inorder predecessor to come back up, eliminating the need for a stack or recursion. O(n) time, O(1) space.

C++

```
1  vector<int> morrisInorder(TreeNode* root) {
2      vector<int> res;
3      TreeNode* curr = root;
4      while (curr) {
5          if (!curr->left) {
6              res.push_back(curr->val); // no left child - visit and go right
7              curr = curr->right;
8          } else {
9              // Find inorder predecessor (rightmost in left subtree)
10             TreeNode* pred = curr->left;
11             while (pred->right && pred->right != curr)
12                 pred = pred->right;
13             if (!pred->right) {
14                 pred->right = curr; // thread: predecessor → curr
15                 curr = curr->left;
16             } else {
17                 pred->right = nullptr; // remove thread
18                 res.push_back(curr->val);
19                 curr = curr->right;
20             }
21         }
22     }
23     return res;
24 }
25 // Time: O(n) Space: O(1) - no stack, no recursion
26 // Each edge traversed at most twice. Tree structure temporarily modified, then restored.
```

4 Binary Tree — Classic Problems

4.1 Height / Maximum Depth (LC 104) — Easy

```
C++  
  
1 // Height = longest path from root to any leaf  
2 int maxDepth(TreeNode* root) {  
3     if (!root) return 0;  
4     return 1 + max(maxDepth(root->left), maxDepth(root->right));  
5 }  
6 // Time: O(n) Space: O(h)  
7  
8 // Minimum depth (to nearest leaf) LC 111  
9 int minDepth(TreeNode* root) {  
10    if (!root) return 0;  
11    if (!root->left) return 1 + minDepth(root->right);  
12    if (!root->right) return 1 + minDepth(root->left);  
13    return 1 + min(minDepth(root->left), minDepth(root->right));  
14 }  
15 // Edge case: node with one child is NOT a leaf — must recurse on non-null side
```

4.2 Diameter of Binary Tree (LC 543) — Easy

Diameter = longest path between any two nodes (not necessarily through root). For each node, the longest path through it = left height + right height. Track global maximum while computing heights bottom-up.

```
C++  
  
1 int diameterOfBinaryTree(TreeNode* root) {  
2     int diameter = 0;  
3     function<int(TreeNode*)> height = [&](TreeNode* node) -> int {  
4         if (!node) return 0;  
5         int L = height(node->left);  
6         int R = height(node->right);  
7         diameter = max(diameter, L + R); // path through this node  
8         return 1 + max(L, R); // height of this subtree  
9     };  
10    height(root);  
11    return diameter;  
12 }  
13 // Time: O(n) Space: O(h)  
14 // Key: update diameter INSIDE the height recursion — avoids O(n^2) naive approach
```

4.3 Balanced Binary Tree (LC 110) — Easy

A tree is balanced if for every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$. Return -1 as a sentinel for 'unbalanced' to avoid redundant recomputation.

```
C++  
  
1  bool isBalanced(TreeNode* root) {  
2      function<int(TreeNode*)> check = [&](TreeNode* node) -> int {  
3          if (!node) return 0;  
4          int L = check(node->left);  
5          if (L == -1) return -1;           // already unbalanced  
6          int R = check(node->right);  
7          if (R == -1) return -1;  
8          if (abs(L - R) > 1) return -1;    // this node is unbalanced  
9          return 1 + max(L, R);           // return height  
10     };  
11     return check(root) != -1;  
12 }  
13 // Time: O(n) - each node visited once  
14 // Naive O(n^2): compute height separately for each node - avoid this
```

4.4 Invert Binary Tree (LC 226) — Easy

```
C++  
  
1  TreeNode* invertTree(TreeNode* root) {  
2      if (!root) return nullptr;  
3      swap(root->left, root->right);      // swap children  
4      invertTree(root->left);  
5      invertTree(root->right);  
6      return root;  
7  }  
8  // Time: O(n) Space: O(h)  
9  
10 // Iterative using queue (BFS)  
11 TreeNode* invertTreeBFS(TreeNode* root) {  
12     if (!root) return root;  
13     queue<TreeNode*> q; q.push(root);  
14     while (!q.empty()) {  
15         TreeNode* node = q.front(); q.pop();  
16         swap(node->left, node->right);  
17         if (node->left) q.push(node->left);  
18         if (node->right) q.push(node->right);  
19     }  
20     return root;  
21 }
```

4.5 Same Tree & Subtree (LC 100, 572)

```

C++
1 // LC 100 - Same Tree
2 bool isSameTree(TreeNode* p, TreeNode* q) {
3     if (!p && !q) return true; // both null
4     if (!p || !q) return false; // one null
5     return p->val == q->val
6         && isSameTree(p->left, q->left)
7         && isSameTree(p->right, q->right);
8 }
9 // Time: O(min(m,n)) Space: O(min(h1,h2))
10
11 // LC 572 - Subtree of Another Tree
12 bool isSubtree(TreeNode* root, TreeNode* subRoot) {
13     if (!root) return false;
14     if (isSameTree(root, subRoot)) return true;
15     return isSubtree(root->left, subRoot)
16         || isSubtree(root->right, subRoot);
17 }
18 // Time: O(m*n) naive - O(m+n) with hashing (advanced)

```

4.6 Count Complete Tree Nodes (LC 222) — Medium

Complete tree: all levels full except last, filled left to right. Exploit structure: if left and right heights equal, left subtree is perfect $\rightarrow 2^h - 1$ nodes + recurse right only.

```

C++
1 int countNodes(TreeNode* root) {
2     if (!root) return 0;
3     // Compute leftmost and rightmost heights
4     int lh = 0, rh = 0;
5     TreeNode* l = root, *r = root;
6     while (l) { lh++; l = l->left; }
7     while (r) { rh++; r = r->right; }
8     if (lh == rh) // perfect subtree
9         return (1 << lh) - 1; // 2^h - 1 nodes
10    return 1 + countNodes(root->left) + countNodes(root->right);
11 }
12 // Time: O(log^2 n) Space: O(log n)
13 // Naive O(n) - this exploits completeness for better complexity

```

5 Binary Tree — Views & Boundary Traversal

5.1 Right Side View (LC 199) — Medium

Return the values visible from the right side (rightmost node of each level). BFS level-order: take the last node of each level.

C++

```
1  vector<int> rightSideView(TreeNode* root) {
2      if (!root) return {};
3      vector<int> res;
4      queue<TreeNode*> q; q.push(root);
5      while (!q.empty()) {
6          int sz = q.size();
7          for (int i = 0; i < sz; i++) {
8              TreeNode* node = q.front(); q.pop();
9              if (i == sz - 1) res.push_back(node->val); // last of level
10             if (node->left) q.push(node->left);
11             if (node->right) q.push(node->right);
12         }
13     }
14     return res;
15 }
16 // Left side view: take i == 0 instead of i == sz-1
17 // Time: O(n) Space: O(w)
```

5.2 Vertical Order Traversal (LC 987) — Hard

Assign (col, row) to each node. Nodes at same (col, row) sorted by value. Use BFS with queue of (node, row, col).

C++

```
1  #include <map>
2
3  vector<vector<int>> verticalTraversal(TreeNode* root) {
4      // map: col -> map: row -> sorted values
5      map<int, map<int, vector<int>>> colMap;
6      queue<tuple<TreeNode*,int,int>> q; // {node, row, col}
7      q.push({root, 0, 0});
8      while (!q.empty()) {
9          auto [node, row, col] = q.front(); q.pop();
10         colMap[col][row].push_back(node->val);
11         if (node->left) q.push({node->left, row+1, col-1});
12         if (node->right) q.push({node->right, row+1, col+1});
13     }
14     vector<vector<int>> res;
15     for (auto& [col, rowMap] : colMap) {
16         vector<int> col_vals;
17         for (auto& [row, vals] : rowMap) {
18             sort(vals.begin(), vals.end()); // same position: sort by value
19             col_vals.insert(col_vals.end(), vals.begin(), vals.end());
20         }
21         res.push_back(col_vals);
22     }
23     return res;
24 }
25 // Time: O(n log n) Space: O(n)
```

5.3 Top View & Bottom View of Binary Tree

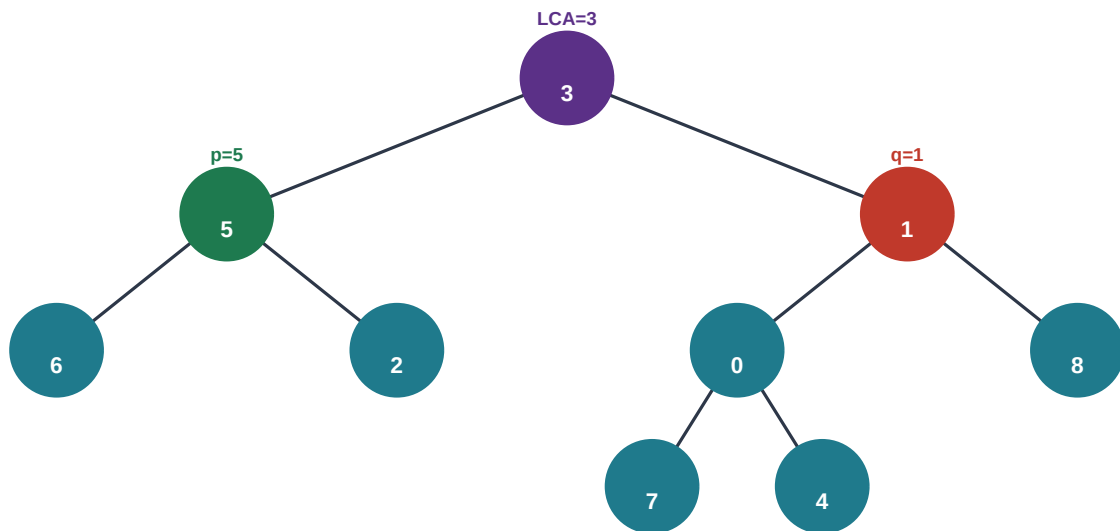
C++

```
1 // Top View: first node seen at each column (smallest row per column)
2 vector<int> topView(TreeNode* root) {
3     if (!root) return {};
4     map<int, int> colFirst;           // col -> first (topmost) node val
5     queue<pair<TreeNode*, int>> q;
6     q.push({root, 0});
7     while (!q.empty()) {
8         auto [node, col] = q.front(); q.pop();
9         if (!colFirst.count(col))    // first time seeing this column
10             colFirst[col] = node->val;
11         if (node->left) q.push({node->left, col-1});
12         if (node->right) q.push({node->right, col+1});
13     }
14     vector<int> res;
15     for (auto& [col, val] : colFirst) res.push_back(val);
16     return res;
17 }
18
19 // Bottom View: LAST node seen at each column (largest row per column)
20 // Same as top view but overwrite (don't check !colFirst.count)
21 vector<int> bottomView(TreeNode* root) {
22     map<int, int> colLast;
23     queue<pair<TreeNode*, int>> q;
24     if (root) q.push({root, 0});
25     while (!q.empty()) {
26         auto [node, col] = q.front(); q.pop();
27         colLast[col] = node->val;    // always overwrite
28         if (node->left) q.push({node->left, col-1});
29         if (node->right) q.push({node->right, col+1});
30     }
31     vector<int> res;
32     for (auto& [col, val] : colLast) res.push_back(val);
33     return res;
34 }
35 // Both: Time O(n log n) Space O(n)
```

6 Lowest Common Ancestor (LCA)

6.1 LCA in Binary Tree (LC 236) — Medium

$LCA(p, q)$ = deepest node that is an ancestor of both p and q . Recursive strategy: if current node is p or q , return it. If p is in left and q is in right (or vice versa), current node is LCA.



$LCA(5, 1) = 3$ | $LCA(5, 4) = 5$ (ancestor of itself)

C++

```
1  TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
2      if (!root) return nullptr;
3      // If current node is p or q, it is a candidate for LCA
4      if (root == p || root == q) return root;
5      TreeNode* left = lowestCommonAncestor(root->left, p, q);
6      TreeNode* right = lowestCommonAncestor(root->right, p, q);
7      // p and q found in different subtrees → root is LCA
8      if (left && right) return root;
9      // One side has both p and q → return that side
10     return left ? left : right;
11 }
12 // Time: O(n) Space: O(h)
13 // This also handles the case where one node is ancestor of the other
```

6.2 LCA in BST (LC 235) — Easy

In a BST, LCA is much simpler: if both p and q are less than root, go left. If both greater, go right. Otherwise, root is the LCA.

C++

```

1  TreeNode* lowestCommonAncestorBST(TreeNode* root, TreeNode* p, TreeNode* q) {
2      while (root) {
3          if (p->val < root->val && q->val < root->val)
4              root = root->left;           // both in left subtree
5          else if (p->val > root->val && q->val > root->val)
6              root = root->right;          // both in right subtree
7          else
8              return root;                 // split point – this is LCA
9      }
10     return nullptr;
11 }
12 // Time:  $O(h) = O(\log n)$  balanced   Space:  $O(1)$  iterative

```

6.3 LCA with Parent Pointers

C++

```

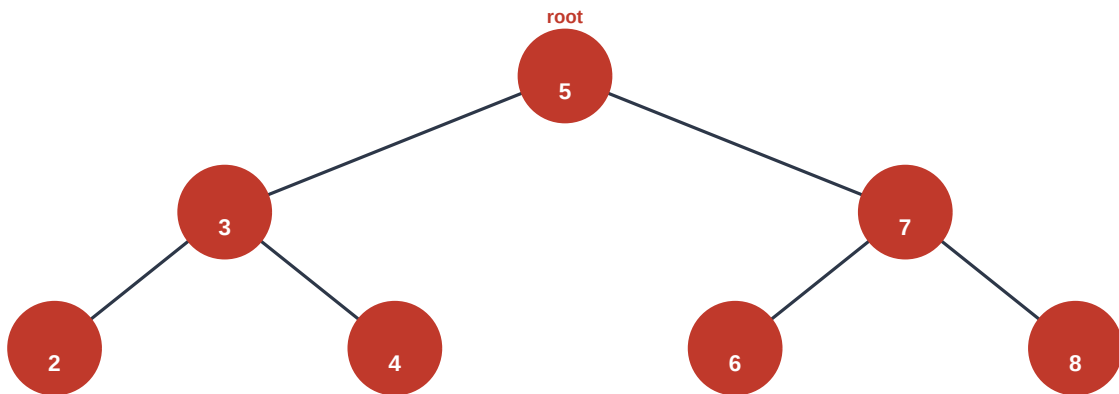
1  // If each node has a parent pointer – use two-pointer trick like intersection of linked lists
2  // Or: collect ancestors of p in a set, traverse ancestors of q until one is in the set
3
4  TreeNode* lcaWithParent(TreeNode* p, TreeNode* q) {
5      unordered_set<TreeNode*> ancestors;
6      // Collect all ancestors of p
7      TreeNode* curr = p;
8      while (curr) { ancestors.insert(curr); curr = curr->parent; }
9      // First ancestor of q found in p's ancestor set = LCA
10     curr = q;
11     while (curr) {
12         if (ancestors.count(curr)) return curr;
13         curr = curr->parent;
14     }
15     return nullptr;
16 }
17 // Time:  $O(h)$    Space:  $O(h)$ 
18 // Two-pointer approach (no set): advance deeper pointer until depths equal,
19 // then advance both until they meet –  $O(h)$  time,  $O(1)$  space.

```

7 Binary Search Tree — Full Operations

7.1 BST Property & Validation

BST invariant: for every node, **all left descendants < node < all right descendants**. This must hold for the entire subtree, not just direct children. Track valid range [minVal, maxVal] as you recurse.



Valid BST: inorder gives sorted sequence 2,3,4,5,6,7,8

C++

```
1 // LC 98 - Validate BST
2 bool isValidBST(TreeNode* root) {
3     function<bool(TreeNode*, long, long)> valid =
4         [&](TreeNode* node, long minV, long maxV) -> bool {
5             if (!node) return true;
6             if (node->val <= minV || node->val >= maxV) return false;
7             return valid(node->left, minV, node->val)
8                 && valid(node->right, node->val, maxV);
9         };
10    return valid(root, LLONG_MIN, LLONG_MAX);
11 }
12 // Time: O(n) Space: O(h)
13 // Use long long to handle INT_MIN/INT_MAX edge cases
```

7.2 BST Insert (LC 701)

C++

```
1 // Insert: maintain BST property — find correct leaf position
2 TreeNode* insertIntoBST(TreeNode* root, int val) {
3     if (!root) return new TreeNode(val); // base: insert here
4     if (val < root->val)
5         root->left = insertIntoBST(root->left, val);
6     else
7         root->right = insertIntoBST(root->right, val);
8     return root;
9 }
10 // Time: O(h) Space: O(h) recursive
11
12 // Iterative version — O(h) time, O(1) space
13 TreeNode* insertIterative(TreeNode* root, int val) {
14     TreeNode* node = new TreeNode(val);
15     if (!root) return node;
16     TreeNode* curr = root;
17     while (true) {
18         if (val < curr->val) {
19             if (!curr->left) { curr->left = node; break; }
20             curr = curr->left;
21         } else {
22             if (!curr->right) { curr->right = node; break; }
23             curr = curr->right;
24         }
25     }
26     return root;
27 }
```

7.3 BST Delete (LC 450) — Medium

Three cases: (1) Node is leaf → delete directly. (2) Node has one child → replace with child. (3) Node has two children → replace value with inorder successor (leftmost of right subtree), then delete successor.

C++

```
1  TreeNode* deleteNode(TreeNode* root, int key) {
2      if (!root) return nullptr;
3      if (key < root->val) {
4          root->left = deleteNode(root->left, key);
5      } else if (key > root->val) {
6          root->right = deleteNode(root->right, key);
7      } else {
8          // Found node to delete
9          if (!root->left) return root->right; // case 1 & 2
10         if (!root->right) return root->left; // case 2
11         // Case 3: two children – find inorder successor
12         TreeNode* succ = root->right;
13         while (succ->left) succ = succ->left; // leftmost of right subtree
14         root->val = succ->val; // copy successor value
15         root->right = deleteNode(root->right, succ->val); // delete successor
16     }
17     return root;
18 }
19 // Time: O(h) Space: O(h)
```

7.4 Kth Smallest Element in BST (LC 230) — Medium

C++

```
1  // Inorder of BST = sorted array → kth element = kth inorder node
2  int kthSmallest(TreeNode* root, int k) {
3      int count = 0, result = 0;
4      function<void(TreeNode*)> inorder = [&](TreeNode* node) {
5          if (!node || count >= k) return;
6          inorder(node->left);
7          count++;
8          if (count == k) { result = node->val; return; }
9          inorder(node->right);
10     };
11     inorder(root);
12     return result;
13 }
14 // Time: O(h + k) Space: O(h) – stops early when kth found
15
16 // Follow-up: if tree is frequently modified, augment each node with subtree size.
17 // Then kth smallest in O(log n) using rank queries.
```

7.5 Floor and Ceil in BST

C++

```
1 // Floor: largest value <= key
2 int floorBST(TreeNode* root, int key) {
3     int floor = -1;
4     while (root) {
5         if (root->val == key) return root->val;
6         if (root->val < key) {
7             floor = root->val; // candidate: could be floor
8             root = root->right; // try to find larger value <= key
9         } else {
10            root = root->left; // too large, go left
11        }
12    }
13    return floor;
14 }
15
16 // Ceil: smallest value >= key
17 int ceilBST(TreeNode* root, int key) {
18     int ceil = -1;
19     while (root) {
20         if (root->val == key) return root->val;
21         if (root->val > key) {
22             ceil = root->val; // candidate
23             root = root->left; // try to find smaller value >= key
24         } else {
25             root = root->right;
26         }
27     }
28    return ceil;
29 }
30 // Both: Time O(h) Space O(1)
```

8 BST — Advanced Problems

8.1 Convert Sorted Array to BST (LC 108) — Easy

Always pick the middle element as root to ensure height-balanced BST. Recursively build left and right subtrees from left and right halves.

C++

```
1  TreeNode* sortedArrayToBST(vector<int>& nums) {
2      function<TreeNode*(int,int)> build = [&](int l, int r) -> TreeNode* {
3          if (l > r) return nullptr;
4          int mid = l + (r - l) / 2;
5          TreeNode* node = new TreeNode(nums[mid]);
6          node->left = build(l,      mid - 1);
7          node->right = build(mid+1, r      );
8          return node;
9      };
10     return build(0, nums.size()-1);
11 }
12 // Time: O(n) Space: O(log n) stack
```

8.2 BST to Greater Sum Tree (LC 1038) — Medium

Replace each node's value with the sum of all values greater than it. Key: reverse inorder (Right → Root → Left) accumulates the running suffix sum.

C++

```
1  TreeNode* bstToGst(TreeNode* root) {
2      int runningSum = 0;
3      // Reverse inorder: Right → Root → Left
4      function<void(TreeNode*)> revInorder = [&](TreeNode* node) {
5          if (!node) return;
6          revInorder(node->right);
7          runningSum += node->val;
8          node->val = runningSum;      // replace with suffix sum
9          revInorder(node->left);
10     };
11     revInorder(root);
12     return root;
13 }
14 // Time: O(n) Space: O(h)
```

8.3 Recover BST (LC 99) — Hard

Two nodes are swapped — restore the BST. Inorder traversal should be sorted. Find the two nodes where order is violated. First violation: $\text{prev.val} > \text{curr.val} \rightarrow \text{prev}$ is candidate 1. Second violation: mark curr as candidate 2. Swap their values.

```
C++  
1 void recoverTree(TreeNode* root) {  
2     TreeNode* first = nullptr; // first wrong node  
3     TreeNode* second = nullptr; // second wrong node  
4     TreeNode* prev = nullptr; // previous inorder node  
5  
6     function<void(TreeNode*)> inorder = [&](TreeNode* node) {  
7         if (!node) return;  
8         inorder(node->left);  
9         if (prev && prev->val > node->val) {  
10             if (!first) first = prev; // first violation: prev is out of place  
11             second = node;           // second always updated (last violation)  
12         }  
13         prev = node;  
14         inorder(node->right);  
15     };  
16     inorder(root);  
17     swap(first->val, second->val); // fix the two swapped nodes  
18 }  
19 // Time: O(n) Space: O(h) - O(1) with Morris traversal
```

8.4 Two Sum in BST (LC 653) — Easy

C++

```
1 // Approach 1: inorder to sorted vector + two pointers - O(n) time O(n) space
2 bool findTarget(TreeNode* root, int k) {
3     vector<int> inorder;
4     function<void(TreeNode*)> dfs = [&](TreeNode* node) {
5         if (!node) return;
6         dfs(node->left); inorder.push_back(node->val); dfs(node->right);
7     };
8     dfs(root);
9     int l = 0, r = inorder.size()-1;
10    while (l < r) {
11        int s = inorder[l] + inorder[r];
12        if (s == k) return true;
13        else if (s < k) l++;
14        else r--;
15    }
16    return false;
17 }
18
19 // Approach 2: HashSet during DFS - O(n) time O(n) space
20 bool findTargetSet(TreeNode* root, int k) {
21     unordered_set<int> seen;
22     function<bool(TreeNode*)> dfs = [&](TreeNode* node) -> bool {
23         if (!node) return false;
24         if (seen.count(k - node->val)) return true;
25         seen.insert(node->val);
26         return dfs(node->left) || dfs(node->right);
27     };
28     return dfs(root);
29 }
```

9 Serialisation & Tree Construction

9.1 Serialize & Deserialize Binary Tree (LC 297) — Hard

Convert tree to string and back. Use preorder traversal with null markers. On deserialise, use an index pointer advancing through the tokens.

```
C++  
  
1  #include <sstream>  
2  using namespace std;  
3  
4  class Codec {  
5  public:  
6      // Encode tree to string  
7      string serialize(TreeNode* root) {  
8          if (!root) return "#,";  
9          return to_string(root->val) + ","  
10             + serialize(root->left)  
11             + serialize(root->right);  
12     }  
13  
14     // Decode string to tree  
15     TreeNode* deserialize(string data) {  
16         istringstream ss(data);  
17         return buildTree(ss);  
18     }  
19 private:  
20     TreeNode* buildTree(istringstream& ss) {  
21         string token;  
22         getline(ss, token, ',');  
23         if (token == "#") return nullptr;  
24         TreeNode* node = new TreeNode(stoi(token));  
25         node->left = buildTree(ss);  
26         node->right = buildTree(ss);  
27         return node;  
28     }  
29 };  
30 // Time: O(n) Space: O(n)
```

9.2 Build Tree from Preorder & Inorder (LC 105) — Medium

Key insight: preorder[0] is always the root. Find root in inorder — elements to the left are the left subtree, elements to the right are the right subtree. Recurse.

C++

```
1  TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
2      unordered_map<int,int> inIdx;
3      for (int i = 0; i < (int)inorder.size(); i++)
4          inIdx[inorder[i]] = i;    // O(1) lookup for root position
5
6      function<TreeNode*(int,int,int,int)> build =
7          [&](int preL, int preR, int inL, int inR) -> TreeNode* {
8              if (preL > preR) return nullptr;
9              int rootVal = preorder[preL];
10             int mid      = inIdx[rootVal];    // root index in inorder
11             int leftSize = mid - inL;
12             TreeNode* node = new TreeNode(rootVal);
13             node->left  = build(preL+1, preL+leftSize, inL,  mid-1);
14             node->right = build(preL+leftSize+1, preR, mid+1, inR );
15             return node;
16         };
17     return build(0, preorder.size()-1, 0, inorder.size()-1);
18 }
19 // Time: O(n) Space: O(n) for map
20 // Similarly: Postorder+Inorder – postorder[last] is root, same logic
```

10 Path Sum Problems

10.1 Path Sum I & II (LC 112, 113)

```
C++  
  
1 // LC 112 - Path Sum (root to leaf, does sum exist?)  
2 bool hasPathSum(TreeNode* root, int targetSum) {  
3     if (!root) return false;  
4     if (!root->left && !root->right) // leaf node  
5         return root->val == targetSum;  
6     return hasPathSum(root->left, targetSum - root->val)  
7         || hasPathSum(root->right, targetSum - root->val);  
8 }  
9  
10 // LC 113 - Path Sum II (return all root-to-leaf paths with target sum)  
11 vector<vector<int>> pathSum(TreeNode* root, int targetSum) {  
12     vector<vector<int>> res;  
13     vector<int> path;  
14     function<void(TreeNode*, int)> dfs = [&](TreeNode* node, int rem) {  
15         if (!node) return;  
16         path.push_back(node->val);  
17         if (!node->left && !node->right && rem == node->val)  
18             res.push_back(path); // found valid path  
19         dfs(node->left, rem - node->val);  
20         dfs(node->right, rem - node->val);  
21         path.pop_back(); // backtrack  
22     };  
23     dfs(root, targetSum);  
24     return res;  
25 }  
26 // Both: Time O(n) Space O(h) stack + O(n) result
```

10.2 Path Sum III — Any Path (LC 437) — Medium

Count paths that sum to target (not necessarily root-to-leaf, but must go downward). Use prefix sum technique: at each node, running sum - target = a prefix sum seen before.

C++

```

1  int pathSum(TreeNode* root, int targetSum) {
2      unordered_map<long long, int> prefixCount;
3      prefixCount[0] = 1; // empty prefix
4      int result = 0;
5      function<void(TreeNode*, long long)> dfs = [&](TreeNode* node, long long currSum) {
6          if (!node) return;
7          currSum += node->val;
8          // Paths ending here with sum == targetSum
9          result += prefixCount[currSum - targetSum];
10         prefixCount[currSum]++;
11         dfs(node->left, currSum);
12         dfs(node->right, currSum);
13         prefixCount[currSum]--; // backtrack: remove from prefix map
14     };
15     dfs(root, 0);
16     return result;
17 }
18 // Time: O(n) Space: O(n) for prefix map
19 // Use long long for currSum to avoid overflow with large values

```

10.3 Binary Tree Maximum Path Sum (LC 124) — Hard

Path can start and end at any node. For each node, max path through it = node.val + max(0, gainLeft) + max(0, gainRight). Track global max.

C++

```

1  int maxPathSum(TreeNode* root) {
2      int globalMax = INT_MIN;
3      function<int(TreeNode*)> maxGain = [&](TreeNode* node) -> int {
4          if (!node) return 0;
5          // Max gain from left and right subtrees (ignore negative contributions)
6          int gainL = max(0, maxGain(node->left));
7          int gainR = max(0, maxGain(node->right));
8          // Path through this node (can go both directions)
9          globalMax = max(globalMax, node->val + gainL + gainR);
10         // Return max one-directional gain (can only extend path in one direction)
11         ,
12         return node->val + max(gainL, gainR);
13     };
14     maxGain(root);
15     return globalMax;
16 }
17 // Time: O(n) Space: O(h)
18 // max(0, gain) ensures negative subtrees don't drag down the path

```

11 Tree DP — Patterns & Problems

11.1 What is Tree DP?

Tree DP computes an answer by combining results from child subtrees in a bottom-up manner. Each recursive call returns one or more values representing the **optimal state of the subtree**. The parent uses these to compute its own state. Pattern: define what to return from each subtree, then combine.

■ Tree DP Core Pattern

State design: what does `dfs(node)` return? → (answer, some_subtree_property)

Bottom-up: process children first, then combine at parent.

Global variable: update a global max/min inside the recursive function.

Key difference from path problems: Tree DP may consider paths THROUGH a node (both children).

Template: `auto [Lresult, Lprop] = dfs(left); auto [Rresult, Rprop] = dfs(right); combine.`

11.2 House Robber III (LC 337) — Medium

Rob houses on a tree: cannot rob a node and its direct children. For each node, return (rob_this_node, skip_this_node). `rob = node.val + skip_left + skip_right`. `skip = max(rob_left, skip_left) + max(rob_right, skip_right)`.

C++

```

1  int rob(TreeNode* root) {
2      // Returns {rob_root, skip_root}
3      function<pair<int,int>(TreeNode*)> dfs = [&](TreeNode* node) -> pair<int,int> {
4          if (!node) return {0, 0};
5          auto [robL, skipL] = dfs(node->left);
6          auto [robR, skipR] = dfs(node->right);
7          // Rob this node: cannot rob children
8          int rob_curr = node->val + skipL + skipR;
9          // Skip this node: children can be robbed or skipped (take best)
10         int skip_curr = max(robL, skipL) + max(robR, skipR);
11         return {rob_curr, skip_curr};
12     };
13     auto [rob_root, skip_root] = dfs(root);
14     return max(rob_root, skip_root);
15 }
16 // Time: O(n) Space: O(h)
17 // Classic Tree DP: return 2-element state from each subtree

```

11.3 Diameter via Tree DP (LC 543) — detailed

Already solved in Section 4 — but here we formalise it as Tree DP. `dfs(node)` returns the height of the subtree. Inside `dfs`, we update `global_diameter = max(diameter, heightL + heightR)`. This is the canonical Tree DP template.

C++

```

1 // Tree DP formalisation of Diameter
2 // State: dfs returns height of subtree rooted at node
3 // Transition: height(node) = 1 + max(height(left), height(right))
4 // Answer: max over all nodes of height(left) + height(right)
5
6 int diameterOfBinaryTree(TreeNode* root) {
7     int ans = 0;
8     function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
9         if (!node) return 0;
10        int L = dfs(node->left);    // Tree DP: get left subtree state
11        int R = dfs(node->right);   // Tree DP: get right subtree state
12        ans = max(ans, L + R);      // combine: path through this node
13        return 1 + max(L, R);      // return: height of this subtree
14    };
15    dfs(root);
16    return ans;
17 }

```

11.4 Longest ZigZag Path (LC 1372) — Medium

ZigZag: go left then right alternately (or right then left). dfs(node) returns (maxLenGoingLeft, maxLenGoingRight). If we go left from parent, we inherit the right-going length of the left child (+1).

C++

```

1 int longestZigZag(TreeNode* root) {
2     int ans = 0;
3     // Returns {max_zigzag_if_came_from_left, max_zigzag_if_came_from_right}
4     function<pair<int,int>(TreeNode*)> dfs = [&](TreeNode* node) -> pair<int,int> {
5         if (!node) return {-1, -1}; // -1 so leaf gets length 0
6         auto [ll, lr] = dfs(node->left);
7         auto [rl, rr] = dfs(node->right);
8         // came_from_parent_left → extend right path of left child
9         int goLeft = 1 + lr;    // turn right at this node: use left child's right length
10        int goRight = 1 + rl;    // turn left at this node: use right child's left length
11        ans = max(ans, max(goLeft, goRight));
12        return {goLeft, goRight};
13    };
14    dfs(root);
15    return ans;
16 }
17 // Time: O(n) Space: O(h)

```

11.5 Binary Tree Cameras (LC 968) — Hard

Place minimum cameras so all nodes are monitored. Greedy + Tree DP. State: 0 = not covered, 1 = covered (no camera), 2 = has camera. Place cameras at nodes whose children are not covered.

```
C++
1  int minCameraCover(TreeNode* root) {
2      int cameras = 0;
3      // Returns: 0=not covered, 1=covered no camera, 2=has camera
4      function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
5          if (!node) return 1; // null nodes are 'covered' (don't need camera)
6          int L = dfs(node->left);
7          int R = dfs(node->right);
8          if (L == 0 || R == 0) {
9              cameras++; // child not covered → place camera here
10             return 2;
11         }
12         if (L == 2 || R == 2)
13             return 1; // at least one child has camera → this is covered
14         return 0; // both children covered but no camera → report as uncovered
15     };
16     if (dfs(root) == 0) cameras++; // root itself not covered
17     return cameras;
18 }
19 // Time: O(n) Space: O(h)
20 // Greedy insight: better to place cameras at parents of leaves than at leaves
```

11.6 Distribute Coins in Binary Tree (LC 979) — Medium

Each node has some coins; total coins = total nodes. Each move passes one coin along an edge. Count minimum moves. For each node, 'excess' = (coins in subtree) - (nodes in subtree). Moves = sum of |excess| over all edges.

```
C++
1  int distributeCoins(TreeNode* root) {
2      int moves = 0;
3      // Returns: net flow of coins needed (+ = excess out, - = need import)
4      function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
5          if (!node) return 0;
6          int L = dfs(node->left);
7          int R = dfs(node->right);
8          moves += abs(L) + abs(R); // each unit of flow = one move on that edge
9          return node->val - 1 + L + R; // net excess of this subtree
10     };
11     dfs(root);
12     return moves;
13 }
14 // Time: O(n) Space: O(h)
15 // node->val - 1: each node needs exactly 1 coin, so surplus = val - 1
```

12 Complexity Cheat Sheet & LeetCode Map

12.1 Traversal & Operation Complexity

Operation	BT	BST (balanced)	BST (skewed)	Space
Inorder/Pre/Post traversal	$O(n)$	$O(n)$	$O(n)$	$O(h)$
Level-order (BFS)	$O(n)$	$O(n)$	$O(n)$	$O(w)$
Height / Depth	$O(n)$	$O(\log n)$	$O(n)$	$O(h)$
Search	$O(n)$	$O(\log n)$	$O(n)$	$O(h)$
Insert	$O(n)$	$O(\log n)$	$O(n)$	$O(h)$
Delete	$O(n)$	$O(\log n)$	$O(n)$	$O(h)$
LCA	$O(n)$	$O(\log n)$	$O(n)$	$O(h)$
Kth smallest	$O(n)$	$O(h+k)$	$O(n)$	$O(h)$
Build from sorted array	$O(n)$	$O(n)$	—	$O(n)$
Serialize / Deserialize	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Tree DP (general)	$O(n)$	$O(n)$	$O(n)$	$O(h)$
Morris Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$

Table 2: Tree operation complexities — h =height, w =max width, n =nodes

12.2 Pattern Decision Tree

230	Kth Smallest in BST	Inorder early stop	Medium
235	LCA of BST	BST property guided	Medium
236	LCA of Binary Tree	Post-order search	Medium
297	Serialize/Deserialize Binary Tree	Preorder + null markers	Hard
337	House Robber III	Tree DP (rob, skip) pair	Medium
437	Path Sum III	Prefix sum + DFS	Medium
450	Delete Node in BST	3-case successor replace	Medium
543	Diameter of Binary Tree	Tree DP height global	Easy
572	Subtree of Another Tree	isSame at each node	Easy
701	Insert into BST	BST property navigate	Medium
968	Binary Tree Cameras	Tree DP 3-state greedy	Hard
979	Distribute Coins	Tree DP net flow	Medium
987	Vertical Order Traversal	BFS + sorted map	Hard
1038	BST to Greater Sum Tree	Reverse inorder running sum	Medium
1372	Longest ZigZag Path	Tree DP 2-state pair	Medium

Table 3: 30 LeetCode problems — Trees, BST & Tree DP

■ Golden Rules — Trees, BST & Tree DP

1. Inorder of BST = sorted. Any problem needing 'sorted order' → think inorder traversal.
2. Tree DP template: dfs() returns a state tuple; update global answer inside dfs.
3. For 'any-to-any path' problems (diameter, max path sum): consider gain = max(0, child_gain).
4. Use a DUMMY/sentinel value (-1, INT_MIN) to propagate 'invalid' states up the tree.
5. Build tree from traversals: root is always first in preorder or last in postorder.
6. LCA: if current node is p or q, return it. If p and q split, current is LCA.
7. Level-order problems: snapshot q.size() BEFORE the inner loop to separate levels.
8. BST delete: always replace with INORDER SUCCESSOR (leftmost of right subtree).
9. Validate BST: pass (min, max) range — not just parent. Skewed duplicates fail simple check.
10. Morris traversal: thread right pointer of inorder predecessor to come back — O(1) space.